

Day 6 — Scaling Databases: Replication & Sharding

The big question: **what happens when even the database needs more than one machine?**

1. Why one machine stops being enough

A single database server can only give you so much. Eventually one of three walls gets hit:

- **Storage wall** — the data no longer fits on one disk.
- **Read wall** — too many read queries per second; the CPU/RAM can't keep up.
- **Write wall** — too many writes per second for one machine to absorb.

There are two ways to respond:

Vertical scaling (scale up) — buy a bigger machine: more RAM, faster CPU, bigger disk. Simple, no code changes. But it's expensive, has a hard ceiling (you can't buy an infinitely large computer), and it's still *one* machine — if it dies, everything dies.

Horizontal scaling (scale out) — add more machines and spread the work. Cheaper commodity hardware, effectively no ceiling, and survives single-machine failure. The cost is complexity: now your data lives in more than one place and you have to keep it coherent.

Replication and sharding are the two core tools of horizontal scaling. They solve *different* problems, which is why serious systems use both.

Tool	Problem it solves	One-line idea
Replication	Failure + too many reads of the same data	Keep copies of the same data on multiple machines
Sharding	Data too big / too many writes for one machine	Split the data across machines, each holds a slice

2. Replication

Replication = keeping the same data on multiple machines (replicas).

Two independent payoffs:

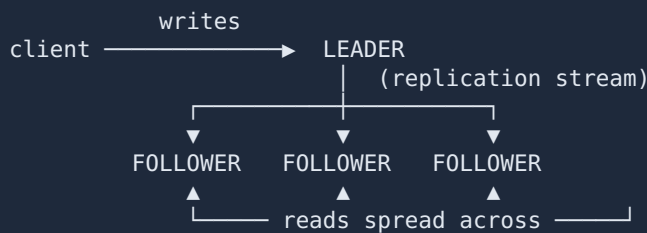
1. **High availability / durability** — if one machine dies, a copy survives. No data loss, no downtime.
2. **Read scaling** — reads can be served by *any* replica, so N replicas $\approx N \times$ read capacity.

Library analogy: stock the same popular novel in ten branches. If one branch floods, the book still exists. And ten people can read it at once instead of queuing for one copy.

2.1 Single-leader replication (leader-follower / master-slave)

The most common setup.

- **One leader** accepts all **writes**.
- The leader streams its changes to one or more **followers**.
- **Reads** can go to the leader *or* any follower.



Synchronous vs asynchronous replication

- **Synchronous** — leader waits for the follower to confirm before telling the client "done." Guarantees the follower is up to date, but a slow follower slows every write.
- **Asynchronous** — leader confirms immediately and ships changes to followers in the background. Fast, but if the leader dies before a change reaches followers, that write is **lost**.
- **Semi-synchronous (common compromise)** — wait for *one* follower synchronously, the rest async.

Replication lag — with async replication, followers are slightly behind the leader. This causes classic bugs:

- **Read-after-write problem** — a user updates their profile (write → leader), then reloads (read → a lagging follower) and sees the *old* data. Fix: route that user's reads to the leader for a short window, or read their own writes from the leader.
- **Monotonic reads** — a user refreshes twice and time appears to go backwards (first read hits an up-to-date follower, second hits a laggier one). Fix: pin a user to the same follower.

Failover — when the leader dies, a follower is promoted to leader. This is the dangerous moment: you must pick the most up-to-date follower, redirect clients, and avoid **split-brain** (two nodes both thinking they're leader). Automatic failover is genuinely hard to get right.

2.2 Multi-leader replication

Multiple leaders, each accepting writes, replicating to each other. Used across data centers (one leader per region) so writes are fast locally.

- **Pro:** writes survive a whole data center going down; low write latency per region.

- **Con:** the same row can be written in two places at once → **write conflicts**. You need conflict resolution (last-write-wins, merge logic, CRDTs). This is the main reason it's avoided unless you truly need it.

2.3 Leaderless replication (Dynamo-style)

No leader. The client (or a coordinator) writes to *several* replicas and reads from *several* at once.

- **Quorums:** with N replicas, require W confirmations to write and R to read. If **W + R > N**, every read overlaps at least one node that saw the latest write → strong-ish consistency.
- Used by Cassandra, DynamoDB, Riak. Great for availability; consistency is tunable, not guaranteed.

3. Sharding (a.k.a. partitioning)

Sharding = splitting the dataset into pieces (shards) and putting each piece on a different machine. Each shard holds a *distinct subset* of the data — no shard has the whole thing.

This is what solves the **storage wall** and the **write wall**: 10 shards means roughly 1/10th the data and 1/10th the writes per machine.

Library analogy: the collection outgrew one building, so cookbooks go to one branch, history to another. Each branch is complete for its subject — a history reader finds *everything* they need in the history building. (Note: splitting one book into Volume 1 / Volume 2 is **not** sharding — that forces a reader to visit two buildings for one thing, which is exactly what a good shard boundary avoids.)

3.1 The shard key

The **shard key** is the column you split on (e.g. `user_id`). Choosing it well is *the* hard decision in sharding, because it determines:

- how evenly data spreads,
- whether most queries hit a single shard or many,
- whether you get hot spots.

Good shard key properties: high cardinality (many distinct values), even access distribution, and it appears in most of your queries.

3.2 Sharding strategies

Range-based sharding — split by ranges of the key (users A-F on shard 1, G-M on shard 2...).

- Pro: range queries are efficient (all in-range data is together).
- Con: easy to create **hot shards**. Sharding by timestamp means all *today's* writes hit one shard while old shards sit idle.

Hash-based sharding — hash the key, use the hash to pick a shard (`shard = hash(user_id) % N`).

- Pro: spreads data evenly, kills hot spots.
- Con: range queries are now scattered across all shards. And plain `% N` is a trap — adding a machine changes `N` and forces almost **all** data to move.
- Fix for that: **consistent hashing**, which only moves a small fraction of keys when you add/remove a node.

Directory / lookup-based sharding — keep a lookup table mapping key → shard.

- Pro: total flexibility; you can move any key anywhere.
- Con: the lookup service becomes a bottleneck and a single point of failure.

Geographic sharding — split by region (EU users in EU, US users in US). Great for latency and data-residency laws (GDPR), but skewed if one region is far bigger.

3.3 The hard parts of sharding

- **Hot shards / skew** — one shard gets disproportionate traffic. Classic case: a celebrity user, or sharding by subject when one subject suddenly goes viral. Mitigation: better key, or split the hot key further (e.g. append a random suffix).
- **Cross-shard queries** — a query that needs data from many shards (a "scatter-gather") is slow and complex, because you hit every shard and merge results.
- **Cross-shard joins and transactions** — joining data on different machines is painful; transactions spanning shards need distributed protocols (like two-phase commit) that are slow and fragile. Design so that data accessed together lives on the same shard.
- **Rebalancing / resharding** — when a shard fills up you must move data to new machines *without downtime*. This is one of the most operationally dangerous tasks in the whole field.

4. Using both together

Sharding and replication cover each other's blind spots:

- **Sharding alone:** you've spread the data, but each shard is a single machine. When shard 3 dies, that slice of data is **gone**.
- **Replication alone:** every machine still holds the *entire* dataset, so you never actually solved "too big for one machine."

The standard architecture: shard first, then replicate each shard.

Data is sharded into 3 slices,
each slice is replicated 3x

SHARD A
(users 0–33%)

leader A
follower
follower

SHARD B
(users 34–66%)

leader B
follower
follower

SHARD C
(users 67–100%)

leader C
follower
follower

Now you get all three benefits at once: the data fits (sharding), writes scale (sharding), and no single machine failure loses data or takes you offline (replication).

5. Consistency: the tax you pay

The moment data lives on more than one machine, you inherit trade-offs.

CAP theorem — during a **network partition** (P — machines can't talk to each other, and this *will* happen), you must choose:

- **CP (consistency)** — refuse requests that can't be made consistent. The system may reject writes / appear down, but never returns wrong data. (e.g. banking, inventory.)
- **AP (availability)** — keep answering, accept that some replicas return stale data, reconcile later. (e.g. social feeds, likes counters.)

You don't choose C vs A in normal operation — only during a partition. The rest of the time you can have both.

Common consistency models (strong → weak):

- **Strong / linearizable** — everyone always sees the latest write. Slowest.
- **Read-your-own-writes** — you always see *your* updates; others might lag.
- **Eventual consistency** — replicas converge "eventually"; reads may be stale for a while. Fast, highly available.

Rule of thumb: use strong consistency where being wrong is unacceptable (money, stock counts), eventual consistency where slight staleness is harmless (view counts, feeds, recommendations).

6. Worked system-design problems

Each problem below states the scenario, the key decision, and a worked answer. Try to answer before reading the solution.

Problem 1 — Design the storage layer for a URL shortener (like bit.ly)

Scenario: Billions of short URLs. Traffic is ~100:1 reads to writes (people click far more than they create). Each record is tiny: `short_code` → `long_url`.

Questions to answer: Shard or replicate first? What's the shard key? Consistency level?

SOLUTION

- **Read-heavy** → replication is your best friend. Add many read replicas so redirects (reads) are served in parallel. Also put a cache (Redis) in front — the hottest URLs get served from memory. - **Billions of rows** → also shard. The dataset won't fit one machine. - **Shard key:** the `short_code` itself, **hash-based**. Lookups are almost always "give me the row for this exact code," which is a single-shard point lookup — perfect for hash sharding. You never do range scans on codes, so hashing's downside doesn't hurt you. - **Consistency:** eventual is fine. A URL created a second ago being briefly missing on one replica is acceptable. Never need cross-shard joins. - **Final:** hash-sharded by `short_code`, each shard replicated for read scale + durability, cache layer on top.

Problem 2 — Design a "post a tweet + read your home timeline" store

Scenario: Users post tweets; each user has a home timeline of tweets from people they follow. Some users have 50 million followers. Reads massively outnumber writes.

Questions: Where's the scaling pain? How do you shard? What breaks with celebrities?

SOLUTION

- **Shard tweets** by `user_id` (hash-based) so one user's tweets live together and writes spread evenly. - **The hard part is the timeline read.** Building a timeline means gathering tweets from everyone you follow — a **cross-shard scatter-gather** across potentially hundreds of shards. Too slow to do on every page load. - **Fan-out on write (the fix):** when you post, push the tweet ID into the precomputed timeline (a cache/list) of each follower. Reads become a single fast lookup of your own list. - **The celebrity problem (hot key + fan-out storm):** a user with 50M followers would trigger 50M writes per tweet. So use a **hybrid**: fan-out-on-write for normal users, but for mega-accounts, *don't* fan out — instead, **pull** their recent tweets at read time and merge them into the timeline. This is a real hot-shard mitigation. - **Consistency:** eventual. A tweet showing up a few seconds late in a timeline is fine.

Problem 3 — Design inventory for an e-commerce flash sale

Scenario: 10,000 units of a hot item, 500,000 people trying to buy in the same minute. You must **never oversell**.

Questions: Can this be eventually consistent? Where does replication hurt? How do you shard?

SOLUTION

- **This is a CP problem.** Overselling = charging people for stock you don't have. Correctness beats availability. Eventual consistency is **not** acceptable for the stock count. - **Replication lag is dangerous here:** if two servers read "1 unit left" off two different replicas, both sell it → oversold. So the decrement must go through a **single source of truth** with an atomic operation (`UPDATE stock SET qty = qty - 1 WHERE qty > 0`) or a transaction / conditional write. Reads-of-the-count for the **decision** go to the leader, not a follower. - **Sharding:** shard by `product_id`, so all traffic for the hot item lands on one shard — which becomes a **hot shard**. Mitigations: keep that item's counter in an in-memory store (Redis atomic `DECR`), or split the 10,000 units into buckets (e.g. 10 counters of 1,000) across shards so the write load spreads, reconciling totals afterward. - **Takeaway:** money/stock ⇒ strong consistency + single-writer semantics for the critical field, even at the cost of throughput.

Problem 4 — Design storage for a chat app (like WhatsApp)

Scenario: Billions of messages. Users open a conversation and read recent messages, scrolling back in time. Huge write volume.

Questions: Shard key? Range or hash? What query pattern drives the choice?

SOLUTION

- **Dominant query:** "get the most recent messages in **this conversation**", then older ones as I scroll." Everything is scoped to a `conversation_id` and ordered by time. - **Shard key:** `conversation_id` so an entire conversation lives on one shard → reading a chat is a **single-shard** query. Never split one conversation across shards (that would recreate the Volume-1/Volume-2 mistake). - **Within a shard, order by timestamp** (range on time) so "recent + scroll back" is a cheap ordered scan. - **Hash the conversation_id across shards** to keep write load even (avoids the "all new messages hit today's shard" hot spot you'd get from sharding on time globally). - **Replicate each shard** for durability — losing chat history is unacceptable — and for read scale. - **Consistency:** read-your-own-writes at minimum (you must see the message you just sent). Others seeing it a beat later is fine.

Problem 5 — When would you deliberately NOT shard?

SOLUTION

Sharding adds serious complexity (cross-shard queries, rebalancing, no easy joins). ****Don't shard until you have to.**** Cheaper first steps, in order: 1. ****Add indexes / optimize queries**** — most "the DB is slow" problems are missing indexes. 2. ****Add a cache**** (Redis/Memcached) in front for hot reads. 3. ****Add read replicas**** — solves read scaling without sharding's pain. 4. ****Vertical scale**** — sometimes a bigger box buys you a year cheaply. 5. ****Only then shard****, when data size or write volume genuinely exceeds one machine. Sharding is a last resort for the write/storage wall, not a default.

7. Quick revision checklist

- Replication = **copies** (fixes failure + read load). Sharding = **splits** (fixes size + write load).
- Single-leader is the default replication setup; watch out for **replication lag** and **failover/split-brain**.
- Sync = safe but slow; async = fast but can lose the last writes.
- Shard key choice is everything: high cardinality, even access, present in most queries.
- Range sharding → good for scans, risks hot shards. Hash sharding → even spread, bad for scans; use **consistent hashing** to avoid mass data movement.
- Biggest sharding headaches: **hot shards, cross-shard joins/transactions, rebalancing**.
- Real systems **shard first, then replicate each shard**.
- **CAP**: during a partition, pick C or A. Money/inventory → CP (strong). Feeds/likes → AP (eventual).
- Don't shard until indexes, caching, replicas, and vertical scaling are exhausted.

One-line summary

Replicate to survive failure and serve more readers; shard to hold more data and absorb more writes; do both because neither alone covers the other's job.